

Fuzzing assistido por IA para contratos Soroban

Um método de auditoria, e a evidência de que ele funciona — medida contra sete bugs plantados, com resposta conhecida.

A IA propõe.
O humano cura.
O fuzzer decide.

Um modelo lê uma expressão e enumera como ela pode dar errado. Ele não sabe dizer se aquele estado é alcançável — e vai afirmar o caso impossível com mais confiança que o real.

A execução é a única autoridade do ciclo que não pode ser convencida de um falso positivo.

Soroban não tem Echidna nem Foundry

O ecossistema Rust entrega a *máquina* — cargo-fuzz, proptest, cargo-mutants — e deixa três trabalhos com o desenvolvedor.

1 · Escrever o harness

Boilerplate. Por função, por contrato.

2 · Saber o que asserir

É aqui que programas de fuzzing morrem.

3 · Escolher onde olhar

Entrada aleatória desperdiça orçamento.

Um fuzzer que só verifica “não deu panic” encontra bugs rasos. As invariantes que valem precisam ser **enunciadas**.

Dois braços, um contrato, sete bugs plantados

soroban-vault: 8 entry points, os três tiers de storage, 12 invariantes, 7 bugs atrás de features individuais. Cada seed foi **provado** quebrar a invariante que alega, antes de qualquer fuzzing.

BRAÇO DE CONTROLE

Escrito da documentação em uma hora

Oráculo: *"a chamada não aborta"*.

Escrito **primeiro**, antes de rodar qualquer prompt.

BRAÇO ASSISTIDO

Invariantes propostas pela IA, curadas à mão

Oráculo: *lê o estado de volta* e compara com um cálculo independente.

Mesmo motor, mesmo contrato, mesmo orçamento. **A única diferença é o oráculo.**

RESULTADO

1 de 7 → 7 de 7

BUGS PLANTADOS DETECTADOS

1/7 → 7/7
controle assistido

ESCORE DE MUTAÇÃO

34% → 98%
controle assistido

COBERTURA DE REGIÕES

85,8% → 96,2%
controle assistido

Mutação é um **instrumento independente**: o cargo-mutants não sabe nada dos bugs plantados. A concordância entre os dois é o que responde à objeção que um benchmark feito sob medida não consegue responder sozinho.

Seis dos sete bugs são silenciosos

BUG PLANTADO	CONTROLE	ASSISTIDO	O QUE ACONTECE
bug_overflow	perdeu	pegou	Previu Overflow, veio ZeroShares
bug_missing_auth	perdeu	pegou	Autorização revogada não rejeitou
bug_zero_amount	perdeu	pegou	Saque de zero teve sucesso
bug_self_transfer	perdeu	pegou	Transferência para si duplicou saldo
bug_no_ttl	perdeu	pegou	TTL da instância não estendido
bug_temp_nonce	pegou	pegou	Único que faz o contrato abortar
bug_reinit	perdeu	pegou	Segundo initialize sobrescreveu o admin
Total	1 / 7	7 / 7	Contrato limpo passa em todos os braços

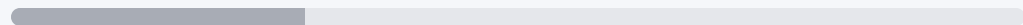
A medição limpa do que a IA acrescenta ao fuzzer

Mesmo alvo, mesmo libFuzzer, mesmos 300 s por célula. A única variável é se o *prior de entradas* gerado pela IA foi incorporado ao gerador.

SEM O PRIOR DE ENTRADAS

2/7

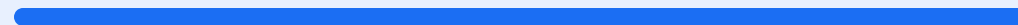
operandos crus do fuzzer



COM O PRIOR DE ENTRADAS

7/7

60% literais de fronteira · 25% relativos ao estado



O prompt tinha sido escrito, executado e commitado — e **nunca aberto**. Um prompt que não é integrado não produz valor e não deixa rastro da própria ausência.

A contribuição foi o oráculo, não o harness

- **13 de 16** propostas aproveitadas — 81% de yield.
- **10 das 12** invariantes de um catálogo que ela nunca viu, re-derivadas só da fonte.
- **4 invariantes novas** que o catálogo não continha.
- As **duas específicas de Soroban** — gestão de TTL e autoridade lida de um tier que expira — derivadas da fonte, incluindo a semântica não óbvia do `extend_ttl`.

O QUE ELA NÃO CONTRIBUIU

16 dos 17 erros de compilação vieram de uma única suposição errada sobre o SDK.

O compilador pegou tudo em segundos. Custo: zero.

Não compre assistência de IA pela geração de código. Compre pela **enumeração de propriedades** — e conte com consertar o código.

A afirmação mais confiante da IA estava **errada**

“o alvo de truncamento de maior valor do arquivo” — `withdraw` pode queimar shares e pagar zero.

Específica, citou as linhas certas, e enunciada com mais confiança que qualquer outra coisa que ela produziu.

É inalcançável.

`assets ≥ total_shares` vale em todo estado alcançável, por indução sobre os quatro mutadores. Com isso, o pagamento nunca é zero.

Ela analisou **uma expressão localmente** e nunca perguntou quais estados são alcançáveis. É onde o checkpoint humano tem que ficar.

A hipótese refutada virou invariante

A afirmação foi rejeitada. Mas a razão de ela ser falsa — `assets ≥ total_shares` — virou a invariante N2, hoje asserida em toda execução.

IA propõe → humano cura → fuzzer decide → volta para a IA.

Refutações são achados.

A cobertura de detecção superou a de invariantes

`bug_zero_amount` tem como alvo a invariante I7 — **que o modelo perdeu** e que não está no conjunto curado.

O braço assistido pega mesmo assim. Em três propriedades diferentes.

Porque o harness prevê a **razão exata da rejeição** para toda entrada — então um guard que para de rejeitar é pego, tenha alguém escrito a invariante ou não.

O argumento mais forte do projeto para investir em **oráculo de erro exato** em vez de alongar a lista de invariantes.

Taxonomias de bug do EVM não se portam

MENOS APLICÁVEL QUE NO EVM

Overflow de inteiro

overflow-checks vem ligado no template Soroban: aritmética ingênua **aborta em vez de dar wrap**. O bug clássico teve que ser plantado com `wrapping_mul` explícito para existir.

SEM ANÁLOGO NO EVM

TTL e tier de storage

Entrada persistente escrita sem estender TTL; autoridade lida de um tier que expira silenciosamente. **Ausentes de todo checklist derivado do EVM** — e é onde mora o risco real de Soroban.

O protocolo 23 auto-restaura entradas persistentes arquivadas. O perigo deixou de ser *perda de dado* e passou a ser **custo de rent inesperado**.

Bug de TTL não é verificável em caixa-preta

Propus dois oráculos “agnósticos de contrato” usando os contadores de recurso do Soroban. Os dois estavam errados — e só descobri porque **medi em vez de supor**.

```
// contrato LIMPO, depois do TTL decair
deposit  write_entries 7  persist_bumps 6

// bug_no_ttl – remove TODOS os extend_ttl do contrato
deposit  write_entries 8  persist_bumps 6  <- idêntico
```

O **host** cobra rent ao escrever uma entrada que iria expirar, o contrato tendo pedido ou não. O contador mede o host, não o contrato.

Detectar `extend_ttl` faltando exige ler o TTL da entrada → exige a chave de storage → exige **ter lido o contrato**.

Seis erros de medição, e o que pegou cada um

O QUE ESTAVA ERRADO	PEGO POR	O QUE TERIA SIDO PUBLICADO
--features aceito e ignorado pela ferramenta	Contradição entre camadas	"o braço de IA perdeu um bug"
Alvo quebrando o contrato limpo	Controle de contrato limpo	8/8, nenhuma atribuível
Asserção nomeando a propriedade errada	Ler texto já marcado verde	Hipótese já refutada, ressuscitada
Oráculo pulando o próprio overflow	Rastrear qual asserção pegou	Detecção perdida como skip
Ledger de curadoria fechando 15 de 16	O mesmo rastreio	Yield 75% em vez de 81%
Script sem bit de execução no git	Gate de clone limpo	Demo que só roda numa máquina

Cinco foram pegos por um gate definido de antemão. O sexto não — **nada exigia reler o texto de uma asserção que já estava verde.**

O que estes números **não** mostram

- **Não é medição de produtividade.** Os dois braços foram escritos pela IA, então o relógio compara um modelo com ele mesmo. A afirmação que este trabalho *não* pode fazer é "IA deixa o dev mais rápido".
- **Não é resultado estatístico.** Um contrato, um modelo, uma rodada por braço. Sem estimativa de variância.
- **Não é livre de viés de benchmark.** O contrato foi construído em torno do catálogo de invariantes, então o modelo lê um contrato cuja forma já implica as propriedades.
- **Duas propriedades de controle de acesso são mais fracas que o enunciado.** "Só o titular autorizou" é testado como "ninguém autorizou" — sinalizado pelo próprio modelo no harness dele.

Um método, e a metade reutilizável dele

soroban-fuzzkit

Política de operandos, pool de principais com os slots de aliasing, sondagem de cliffs de TTL, atribuição de erro. 40 testes, não menciona contrato nenhum.

AUDITING.md

O que remover antes de mostrar o fonte a um modelo, por que o controle de contrato limpo roda primeiro, como separar achado real de achado fabricado.

```
# reproduz em ~2 minutos, de um clone limpo
git clone <repo> && cd stellar-studies
cargo test
./04-prototype-development/scripts/demo.sh
```

EM UMA FRASE

A IA não ajudou escrevendo o harness — ela errou o SDK e o compilador corrigiu em segundos.

Ela ajudou *sabendo o que asserir*, o que levou a detecção de 1 em 7 para 7 em 7 — e a única coisa sobre a qual ela estava mais confiante era a única coisa falsa.

< stellar-studies >